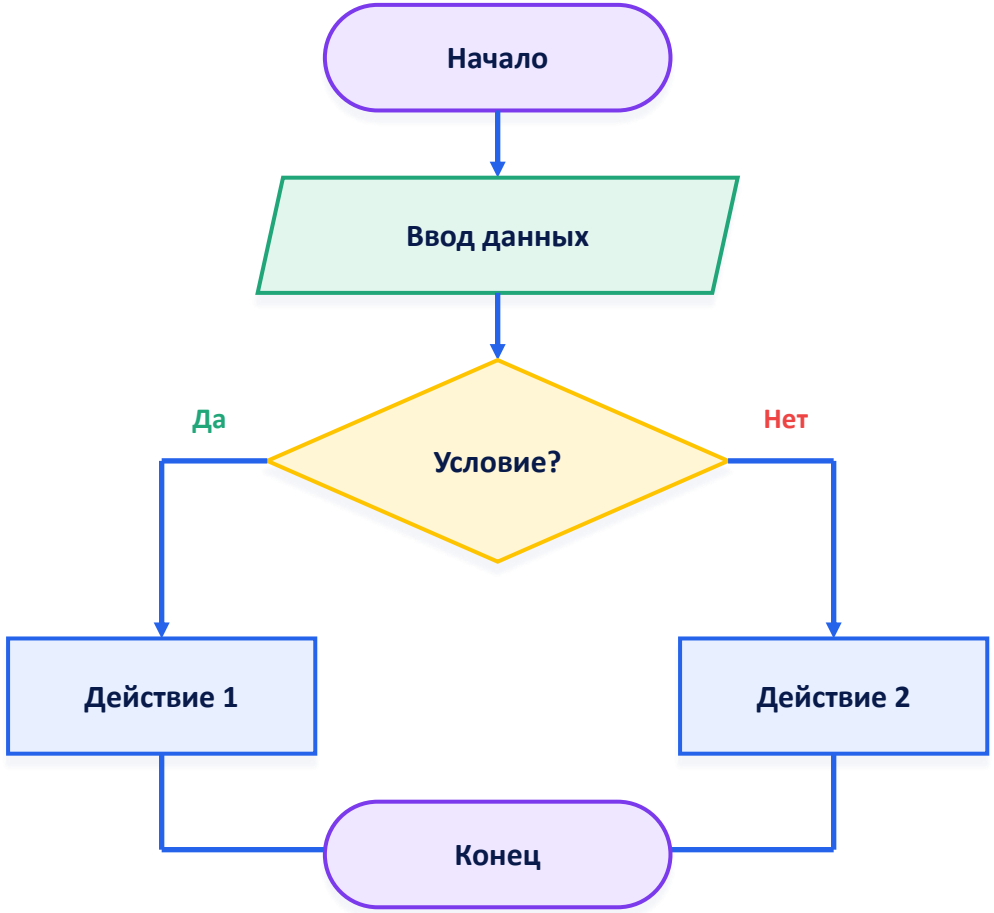
SMART

Алгоритм и программа: от идеи к коду

Как описывать решение задачи так, чтобы его смог выполнить исполнитель — человек или компьютер.

Алгоритм • исполнитель • данные • блок-схема • программа

Урок информатики · 8–11 класс, СПО



Любую задачу можно разложить на шаги, проверки и повторения — а затем записать на Python.

SMART

Алгоритмы уже вокруг нас

Каждый день мы выполняем последовательности действий, даже не замечая этого.

Приготовление еды

1. Взять хлеб.
2. Намазать масло.
3. Положить сыр.

Переход дороги

1. Дождаться зелёного сигнала.
2. Посмотреть по сторонам.
3. Перейти дорогу.

Поиск маршрута

1. Ввести адрес.
2. Сравнить варианты пути.
3. Начать движение.

Алгоритм — это последовательность действий для достижения цели

SMART

Что такое алгоритм?

Не всякий список действий — правильный алгоритм: у него есть пять свойств.

1. **Дискретность:** разбит на отдельные шаги
2. **Точность:** каждый шаг описан однозначно
3. **Конечность:** завершается за конечное число шагов
4. **Результативность:** обязательно даёт результат
5. **Массовость:** применим к классу однотипных задач

Определение

Алгоритм — точное описание последовательности действий, приводящих к решению задачи за конечное число шагов.

Мини-проверка

Какое свойство нарушает команда «выполнять действие бесконечно»?

Ответ: конечность

SMART

Исполнитель понимает только свои команды

Исполнитель — объект, который выполняет команды из своей системы команд.

Человек

Система команд:

- Ходить
- Писать
- Считать

Робот

Система команд:

- Перемещаться
- Брать предметы

Компьютер, умная колонка

Система команд:

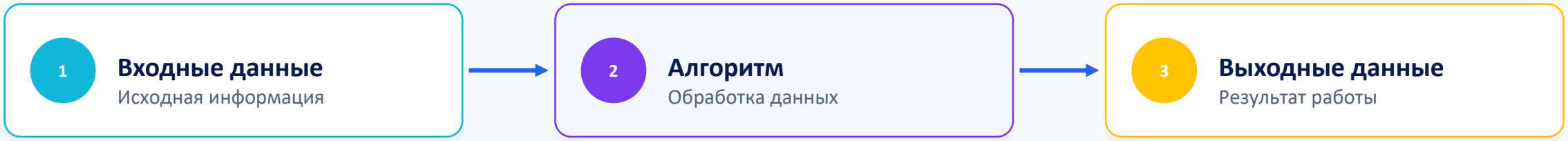
- Выполнять программы
- Хранить данные
- Воспроизводить музыку

! **Калькулятор не может приготовить чай: исполнитель выполняет только те команды, которые входят в его систему команд.**

SMART

Данные: от входа к выходу

Пример: найти площадь прямоугольника со сторонами 5 и 3.



Сторона A = 5

Сторона B = 3

$$S = A \times B$$

Площадь S = 15

Вход поступает извне

Ввод пользователя, файл, датчик — исходные данные задачи.

Выход — итоговый результат

Промежуточные переменные не считаются ни входом, ни выходом.

SMART

Три базовые алгоритмические конструкции

Из них строится любой алгоритм, а в реальных задачах они сочетаются.

Последовательность

Действия выполняются строго друг за другом.

Пример

Шаги приготовления чая

Ветвление

Выбор действия зависит от проверки условия: да или нет.

Пример

Если дождь — взять зонт

Повторение (цикл)

Некоторые действия выполняются несколько раз.

Пример

Пока есть задачи — решать следующую

Внутри цикла может быть условие, а в каждой ветви — последовательность команд

SMART

1. Последовательность: шаг за шагом

Задача: приготовить чай. Каждый шаг выполняется после предыдущего.

1. **Взять чайник.**
2. **Налить воду.**
3. **Включить чайник и дождаться кипения.**
4. **Положить пакетик чая в кружку.**
5. **Налить кипяток и подождать 3 минуты.**

Главное правило

Порядок важен: нельзя налить кипяток до того, как вода закипела. Перестановка шагов меняет результат.

Конечность

Алгоритм заканчивается: чай готов.

Действия выполняются строго по порядку

2. Ветвление: проверка условия

Исполнитель выбирает одну из ветвей и не выполняет команды другой.

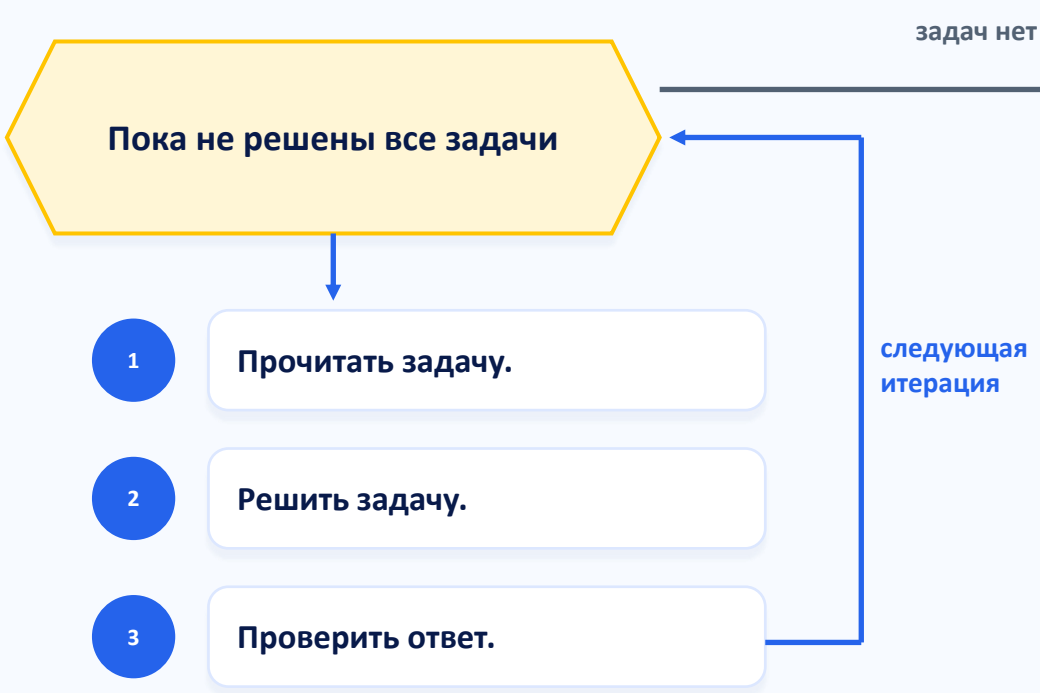


Если [условие истинно], **то** [действие 1], **иначе** [действие 2].

SMART

3. Повторение (цикл): пока условие выполняется

Одно выполнение тела цикла называется итерацией.



Выход из цикла

Все задачи решены — цикл завершается.

Важно

У цикла должно быть условие завершения, иначе он будет повторяться бесконечно.

Условие и цикл — не одно и то же

Условие выбирает ветвь, цикл организует повторение.

SMART

Язык блок-схем: как нарисовать алгоритм

Фигуры соединяются стрелками; чтение начинается с «Начала» и заканчивается «Концом».



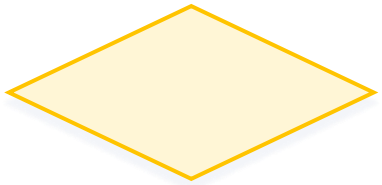
Овал

Начало / конец алгоритма



Прямоугольник

Действие: вычисление, присваивание



Ромб

Условие: ветвление «Да / Нет»



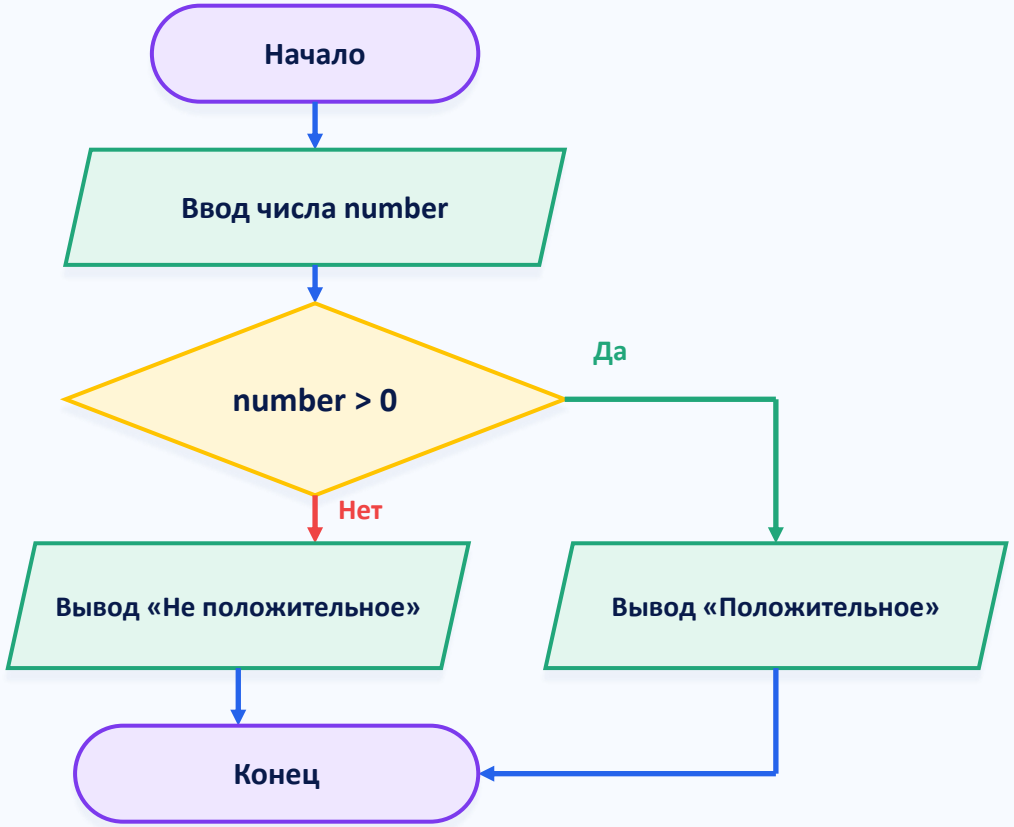
Параллелограмм

Ввод и вывод данных

В ромбе пишут условие («x > 5?»), а команду «увеличить x на 1» — в прямоугольнике

Блок-схема в действии: проверка числа

Определить, является ли введённое число положительным.



Как читать схему

Идём по стрелкам сверху вниз. В ромбе выбираем ветку «Да» или «Нет».

Граничный случай

Число 0 попадёт в ветку «Нет»: условие number > 0 для него ложно, хотя 0 не отрицательный.

SMART

От блок-схемы к программному коду

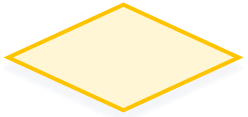
Каждой конструкции алгоритма соответствует конструкция языка Python.



Прямоугольник

действия друг за другом

→ `строки кода сверху вниз`



Ромб

условие, ветвление

→ `if … else …`



Стрелка назад

повторение (цикл)

→ `while … / for …`



Параллелограмм

ввод и вывод данных

→ `input() / print()`

SMART

Код на Python: расчёт скидки

Ввод, ветвление и вывод — те же блоки, что и в блок-схеме.

```
purchase_amount = float(input('Сумма: '))

if purchase_amount > 10000:
    discount = purchase_amount * 0.10
else:
    discount = 0

print('Скидка:', discount, 'руб.')
```

Граница: при сумме ровно 10 000 скидки нет (условие > 10000)

Входные данные

input() + float() — параллелограмм «Ввод».

Ветвление

if … else — ромб: сумма больше 10 000? Тело ветви — с отступом.

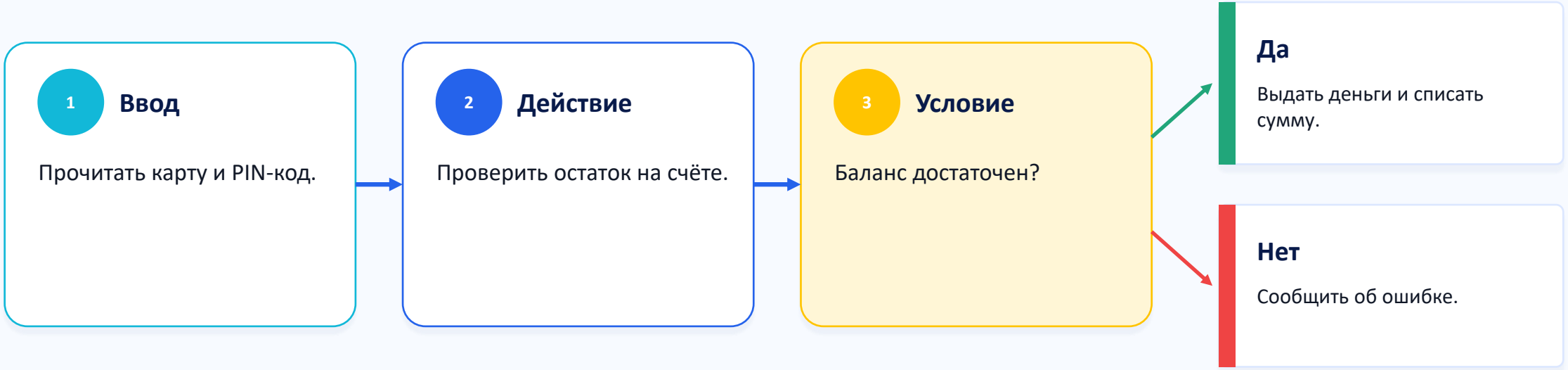
Выходные данные

print() — параллелограмм «Вывод».

SMART

Реальный кейс: алгоритм банкомата

Сложные системы строятся из тех же базовых блоков логики.



Где здесь входные данные?

Карта, PIN-код и запрошенная сумма.

Где выходные?

Выданные деньги или сообщение об ошибке.

Последовательность + ветвление: тот же алгоритм, что и в простых задачах

SMART

Памятка урока: от идеи к коду

Главные понятия занятия.

Понятие	Что означает
Алгоритм	Точная последовательность шагов для решения задачи.
Исполнитель	Тот, кто выполняет команды: человек, робот, компьютер.
Входные данные	Исходная информация, поступающая для решения задачи.
Выходные данные	Результат работы алгоритма.
Блок-схема	Графическое описание алгоритма: овалы, ромбы, прямоугольники.
Программа	Алгоритм, записанный на языке программирования, например на Python.

План решения

Результат → данные → шаги → конструкции → проверка → код

Проверяйте

обычные и граничные значения.